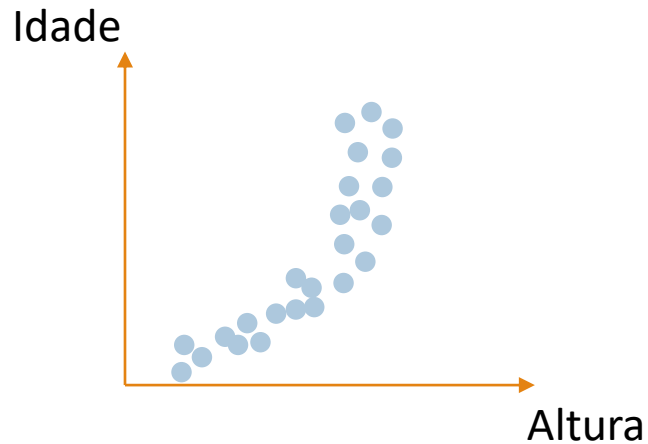


Redes Neurais para Processamento de Imagens

PROF. CESAR HENRIQUE COMIN

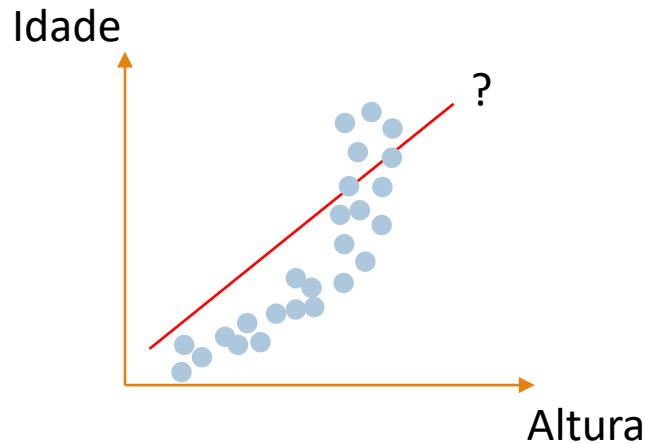
Regressão linear

Vamos supor que queremos prever a idade de uma pessoa com base em algumas de suas características



Regressão linear

Vamos supor que queremos prever a idade de uma pessoa com base em algumas de suas características



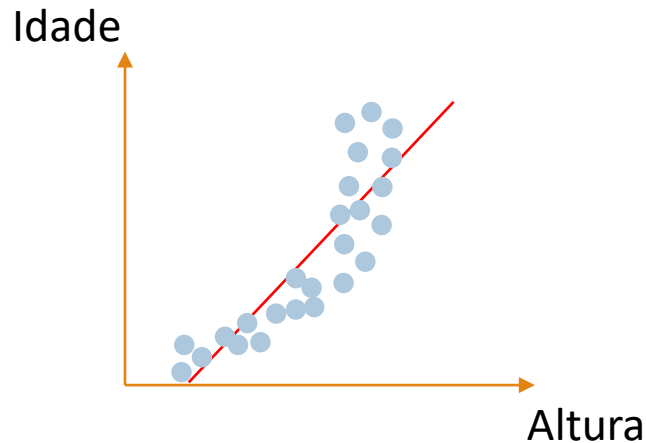
Uma possibilidade: relação linear

$$\text{Idade} = w_1 * \text{Altura} + w_0$$

- Precisamos encontrar w_0 e w_1 que forneça o melhor ajuste de linha reta para a relação entre idade e altura.

Regressão linear

Vamos supor que queremos prever a idade de uma pessoa com base em algumas de suas características



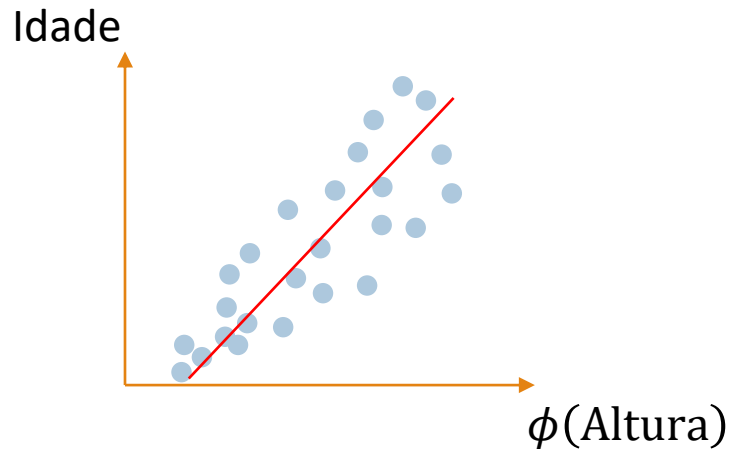
Uma possibilidade: relação linear

$$\text{Idade} = w_1 * \text{Altura} + w_0$$

- Precisamos encontrar w_0 e w_1 que forneça o melhor ajuste de linha reta para a relação entre idade e altura.
- Usamos regressão de mínimos quadrados para encontrar o melhor w_0 e w_1 !

Regressão linear

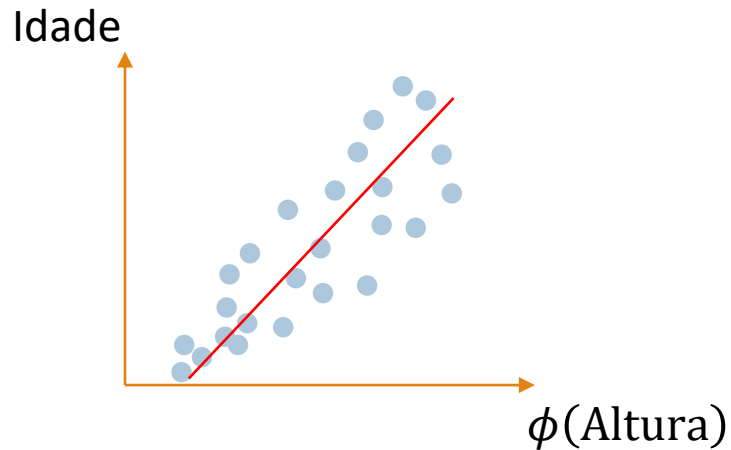
A relação não parece perfeitamente linear, portanto, podemos mapear o valor de altura para um novo valor $\phi(\text{Altura})$, o que, com sorte, tornará o relacionamento linear



Por exemplo
 $\phi(\text{Altura}) = \text{Altura}^2$

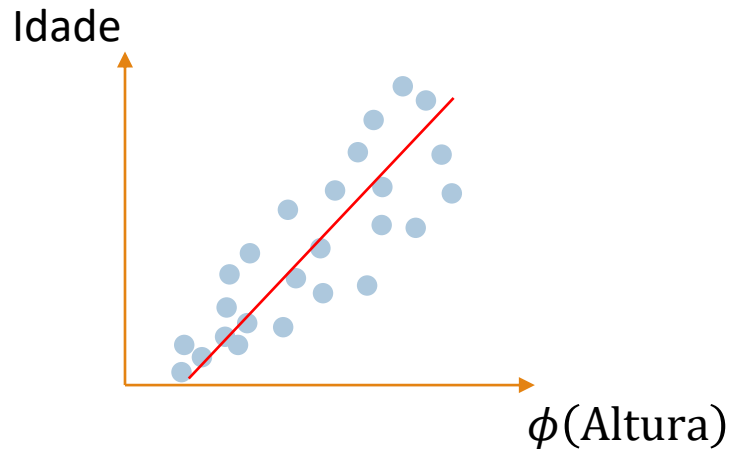
Regressão linear

Temos um ajuste melhor, mas ainda assim, a previsão não é muito boa. Para uma dada altura, a pessoa pode ter uma grande variedade de idades diferentes. Como podemos melhorar isso?



Regressão linear

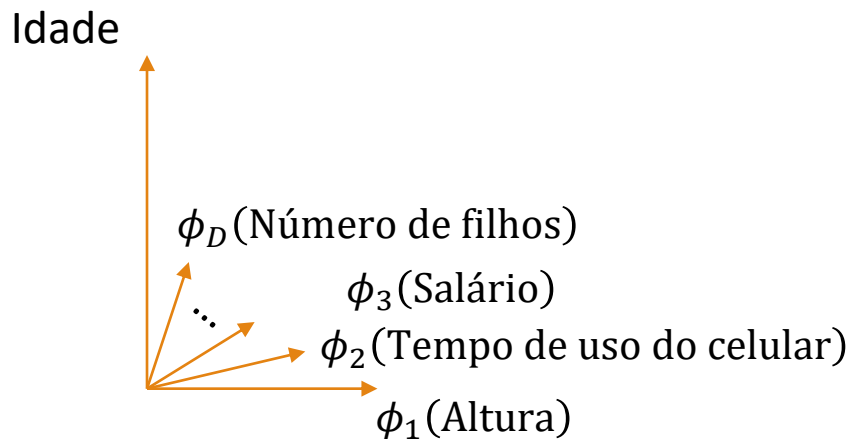
Temos um ajuste melhor, mas ainda assim, a previsão não é muito boa. Para uma dada altura, a pessoa pode ter uma grande variedade de idades diferentes. Como podemos melhorar isso?



Adicionar mais atributos!

Regressão linear

Temos um ajuste melhor, mas ainda assim, a previsão não é muito boa. Para uma dada altura, a pessoa pode ter uma grande variedade de idades diferentes. Como podemos melhorar isso?



$$\text{Idade} = w_1 * \phi_1(\text{Altura}) + w_2 * \phi_2(\text{Tempo de uso do celular}) + w_3 * \phi_3(\text{Salário}) + \dots + \phi_D(\text{Número de filhos}) + w_0$$

Regressão linear

Então, nosso modelo é esse

$$\text{Idade} = w_0 + \sum_{i=1}^D w_i \phi_i(x_i)$$

Mas o modelo pode ser ainda mais flexível. Cada função ϕ_i pode depender não apenas de x_i , mas de todas as variáveis. Escrevemos $\phi_i(x_1, x_2, \dots, x_D)$ como $\phi_i(\mathbf{s})$. Nosso modelo agora é

$$\text{Idade} = w_0 + \sum_{i=1}^M w_i \phi_i(\mathbf{s})$$

* Observe que podemos definir $\phi_i(\mathbf{s}) = x_i$. Nesse caso, temos as características originais

Regressão linear

$$\text{Idade} = w_0 + \sum_{i=1}^M w_i \phi_i(\mathbf{s})$$

Podemos escrever o modelo de uma forma mais compacta

$$\text{Idade} = \mathbf{w}\boldsymbol{\phi}$$

Onde $\mathbf{w} = (w_0, w_1, \dots, w_M)$, $\boldsymbol{\phi} = (\phi_0, \phi_1, \dots, \phi_M)$ e $\phi_0 = 1$.

Ainda podemos usar a regressão de mínimos quadrados para encontrar $w_0, w_1, \dots, w_M$!

Regressão logística

$$\text{Idade} = w\phi$$

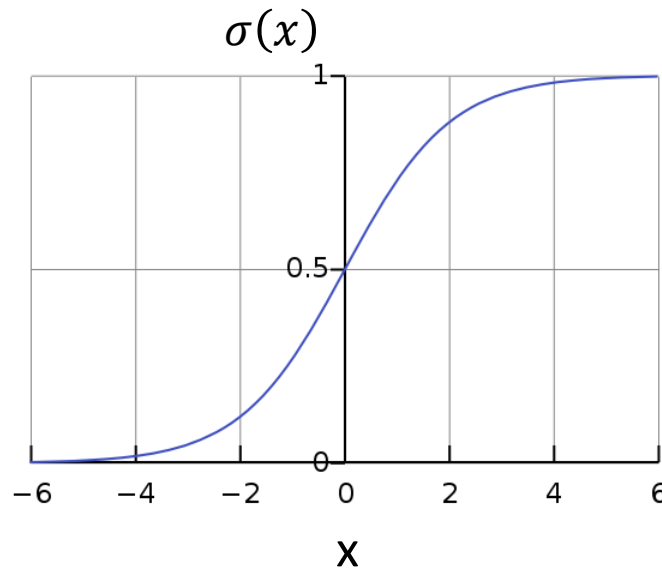
E se quisermos **classificar** uma pessoa em *jovem* ou *idosa* , dependendo de um conjunto de características?

Podemos utilizar a **regressão logística**

Regressão logística

Na regressão logística, mapeamos a combinação linear entre os atributos usando uma função não linear definida entre $[0,1]$, denominada função logística.

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$



Regressão logística

A vantagem desse mapeamento é que podemos relacionar o resultado da função logística à probabilidade de uma determinada pessoa $\mathbf{s}_i$ pertencer à classe jovem ou idosa:

$$P(\text{Idosa}|\mathbf{s}_i) = \sigma(\mathbf{w}\boldsymbol{\phi}(\mathbf{s}_i))$$

Regressão logística

A vantagem desse mapeamento é que podemos relacionar o resultado da função logística à probabilidade de uma determinada pessoa $\mathbf{s}_i$ pertencer à classe jovem ou idosa:

$$P(\text{Idosa}|\mathbf{s}_i) = \sigma(\mathbf{w}\boldsymbol{\phi}(\mathbf{s}_i))$$

$$P(\text{Jovem}|\mathbf{s}_i) = 1 - P(\text{Idosa}|\mathbf{s}_i)$$

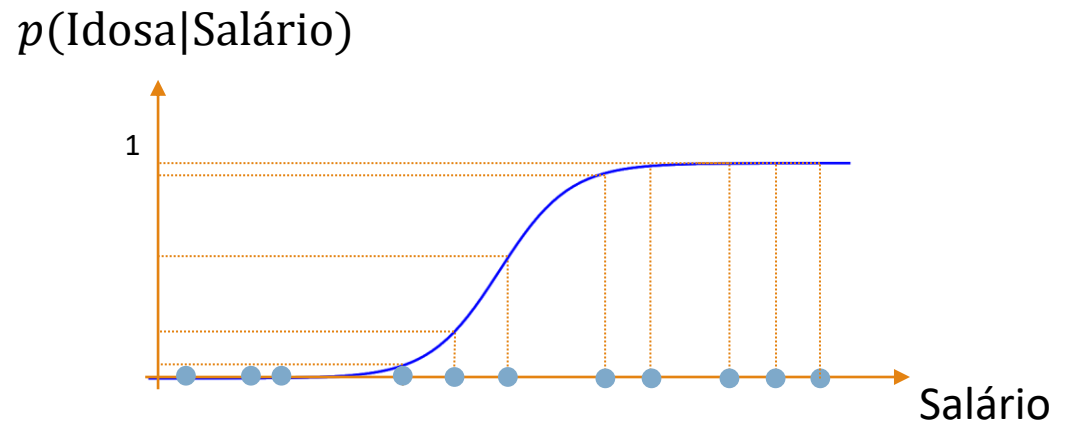
Regressão logística

A vantagem desse mapeamento é que podemos relacionar o resultado da função logística à probabilidade de uma determinada pessoa s_i pertencer à classe jovem ou idosa:

$$P(\text{Idosa}|\mathbf{s}_i) = \sigma(\mathbf{w}\boldsymbol{\phi}(\mathbf{s}_i))$$

$$P(\text{Jovem}|\mathbf{s}_i) = 1 - P(\text{Idosa}|\mathbf{s}_i)$$

Por exemplo, quando
temos apenas uma
variável



Regressão logística

$$P(\text{Idosa}|\mathbf{s}_i) = \sigma(\mathbf{w}\boldsymbol{\phi}(\mathbf{s}_i))$$

Portanto, podemos definir o seguinte critério de decisão:

Se $\sigma(\mathbf{w}\boldsymbol{\phi}(\mathbf{s}_i)) > 0.5$, a pessoa é idosa

Caso contrário, a pessoa é jovem

Regressão logística

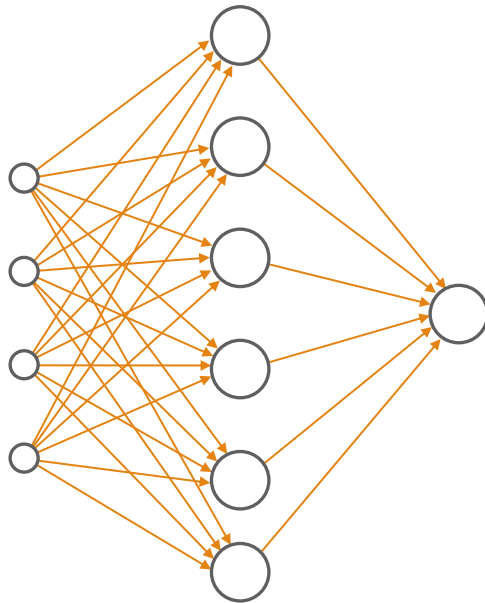
- Treinamos a regressão logística encontrando os parâmetros $\mathbf{w}$ que maximizam a função de verossimilhança do nosso modelo, utilizando os dados que temos sobre um conjunto de pessoas.
- Observe que o treinamento **não maximiza diretamente a acurácia.**

Regressão logística

- E as funções de mapeamento $\phi_i(s)$?
- Até agora, assumimos que funções de mapeamento apropriadas são conhecidas
- Mas geralmente elas não são conhecidas

Regressão logística utilizando redes neurais

- Podemos realizar regressão logística utilizando redes neurais
- Esse modelo é chamado de Perceptron em alguns contextos, apesar de não ser um nome 100% adequado

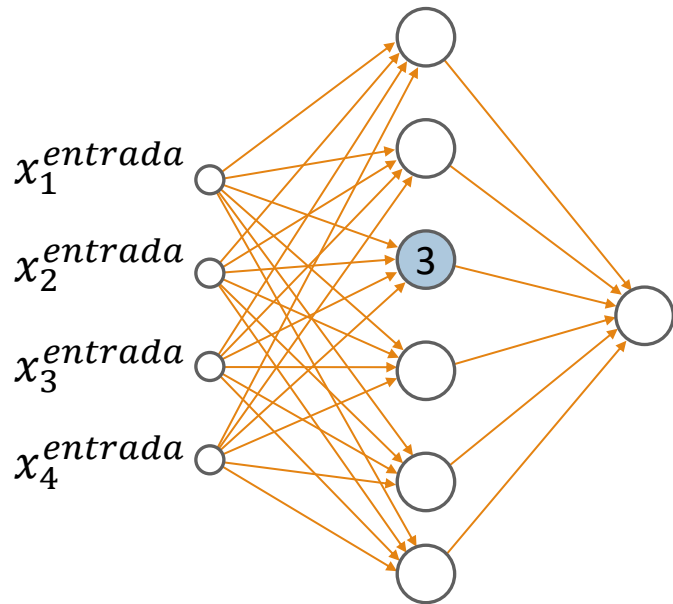


Camada
de entrada
(propriedades)

Camada
escondida

Camada de saída
(probabilidade
jovem/idososa)

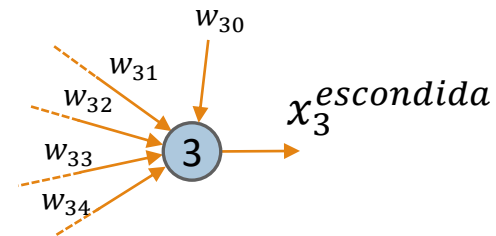
Regressão logística utilizando redes neurais



Camada
de entrada
(propriedades)

Camada
escondida

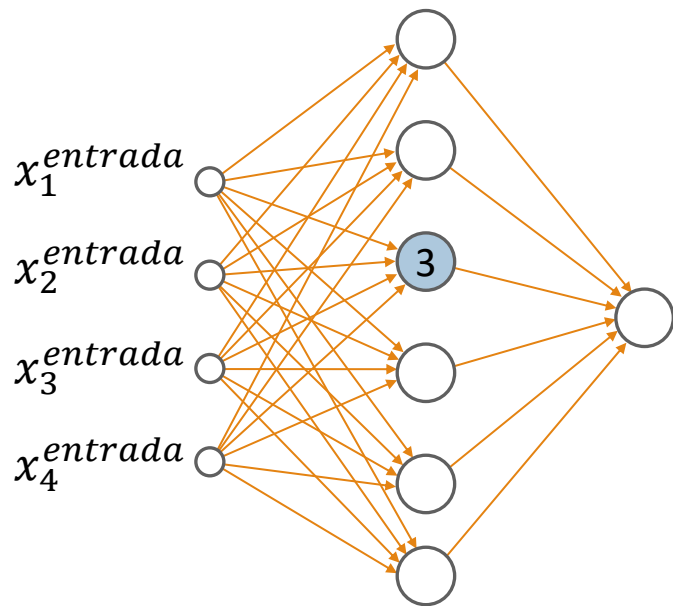
Camada de saída
(probabilidade
jovem/idosa)



Para um dado neurônio j na camada escondida:

$$x_j^{escondida} = S \left(w_{j0} + \sum_{i=1}^4 w_{ji} x_i^{entrada} \right)$$

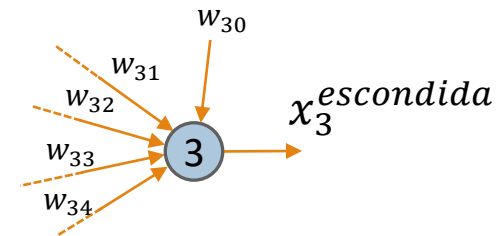
Regressão logística utilizando redes neurais



Camada
de entrada
(propriedades)

Camada
escondida

Camada de saída
(probabilidade
jovem/idosa)



Para um dado neurônio j na camada escondida:

$$x_j^{escondida} = S \left(w_{j0} + \sum_{i=1}^4 w_{ji} x_i^{entrada} \right)$$

Ativação de um neurônio de saída :

$$P(\text{Idosa}) = S \left(w_{o0} + \sum_{j=1}^6 w'_{oj} x_j^{escondida} \right)$$

Regressão logística utilizando redes neurais

- Efetivamente, estamos definindo novas variáveis $x_j^{escondida}$, que são uma função das variáveis de entrada
- Isto é, $x_j^{escondida} = \phi_j(\mathbf{x}^{entrada})$

$$x_j^{escondida} = S\left(w_{j0} + \sum_{i=1}^4 w_{ji} x_i^{entrada}\right)$$

$$P(\text{Idosa}) = S\left(w_{o0} + \sum_{j=1}^6 w'_{oj} x_j^{escondida}\right)$$

Regressão logística utilizando redes neurais

- Efetivamente, estamos definindo novas variáveis $x_j^{escondida}$, que são uma função das variáveis de entrada
- Isto é, $x_j^{escondida} = \phi_j(x^{entrada})$
- Assim, estamos fazendo um mapeamento não linear das variáveis de entrada em, idealmente, variáveis mais apropriadas
- Então, $P(\text{Idosa})$ é dado por uma combinação linear das variáveis ocultas, mapeadas no intervalo $[0,1]$ pela função logística

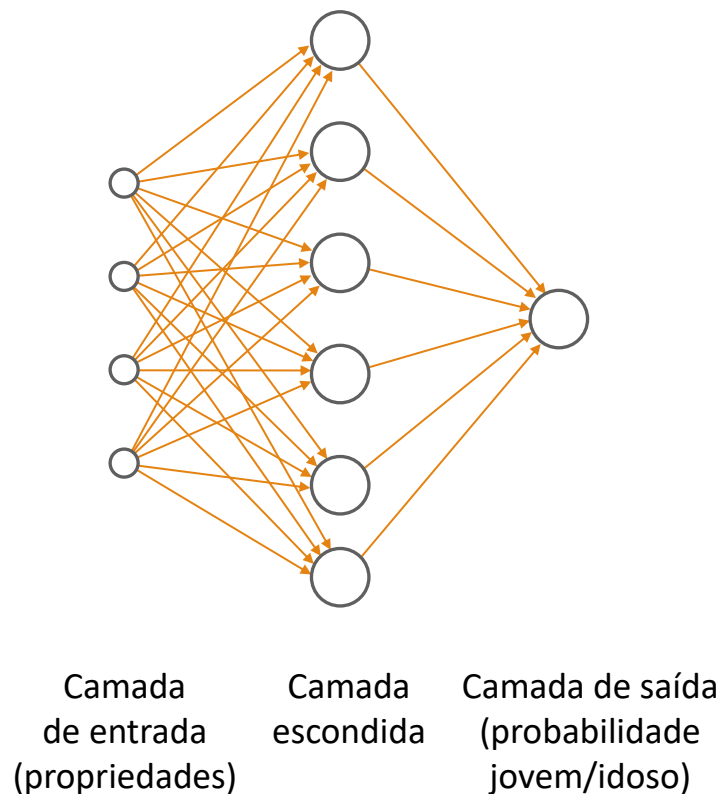
$$x_j^{escondida} = S\left(w_{j0} + \sum_{i=1}^4 w_{ji} x_i^{entrada}\right)$$

* Se quisermos fazer regressão em vez de classificação, removemos $S()$ da saída

$$P(\text{Idosa}) = S\left(w_{o0} + \sum_{j=1}^6 w'_{oj} x_j^{escondida}\right)$$

Regressão logística utilizando redes neurais

Observe que temos que ajustar 37 parâmetros!

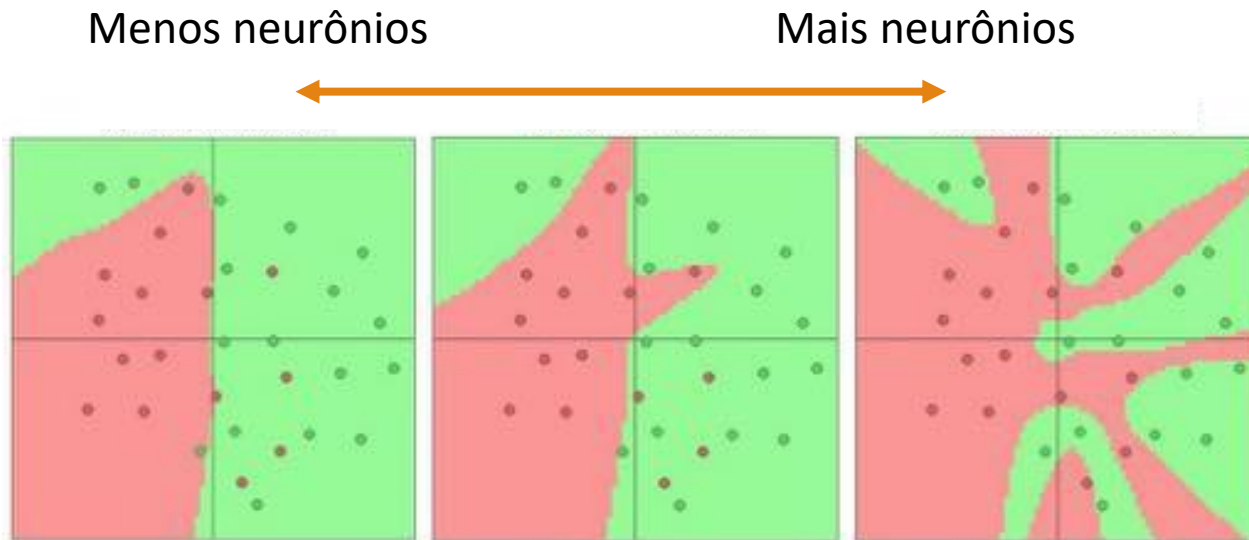


$$x_j^{escondida} = S \left(w_{j0} + \sum_{i=1}^4 w_{ji} x_i^{entrada} \right)$$

$$P(\text{Idosa}) = S \left(w_{o0} + \sum_{j=1}^5 w'_{oj} x_j^{escondida} \right)$$

Regressão logística utilizando redes neurais

O número de camadas ocultas e o número de neurônios nessas camadas está associado à complexidade do modelo representado pela rede neural.



Redes Neurais - Teorema da Aproximação Universal

Um resultado importante sobre redes neurais é o teorema da aproximação universal, que afirma:

Uma rede neural com uma única camada oculta pode aproximar todas* as funções contínuas.

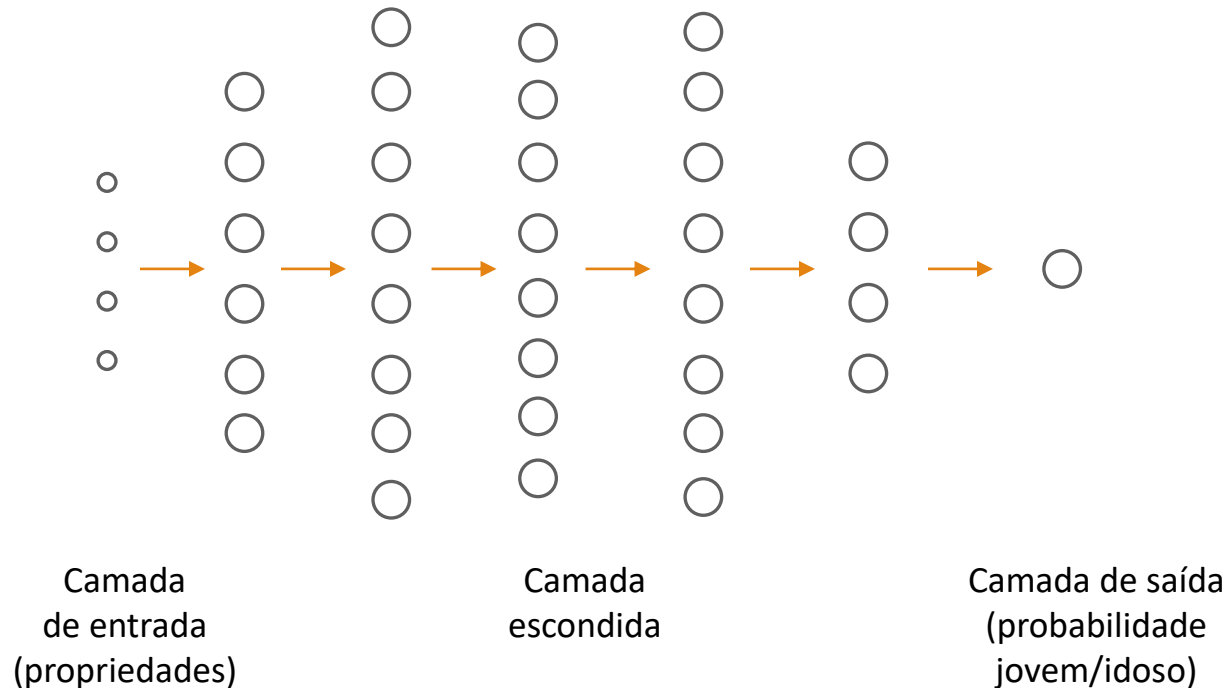
Isso significa que qualquer função da forma

$$y = f(x)$$

pode ser representada como uma rede neural.

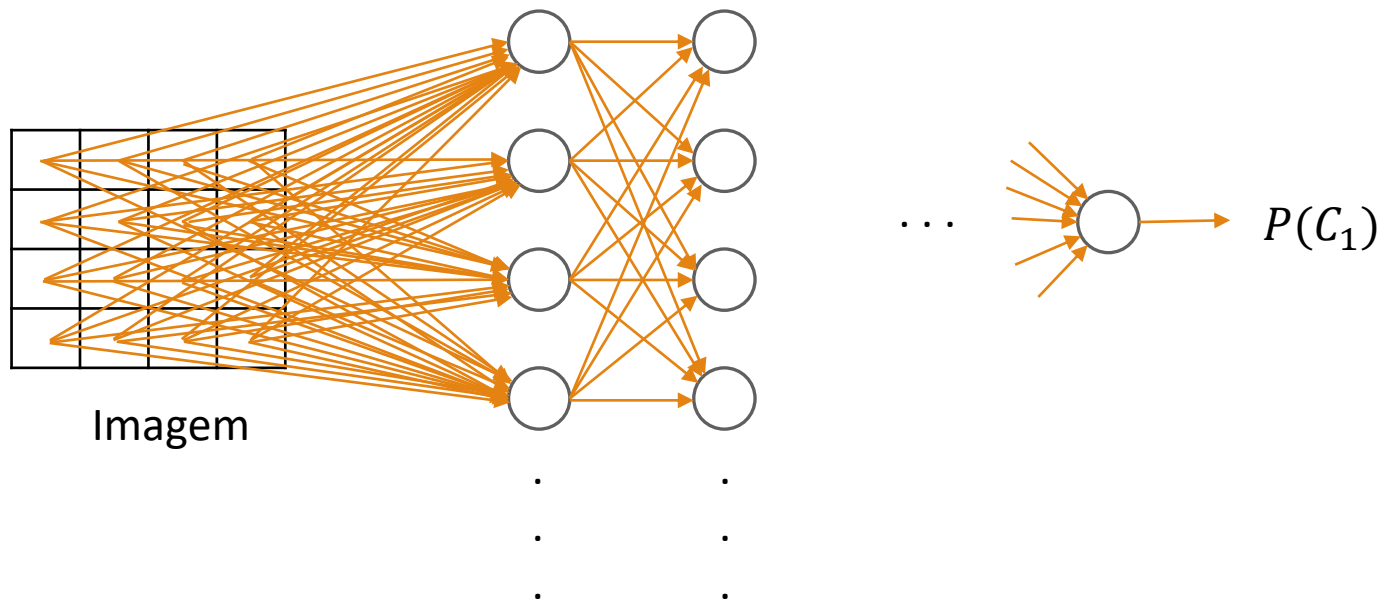
Aprendizado profundo

- Uma rede neural com muitas camadas pode ser chamada de rede profunda. O processo de treinamento de uma rede profunda é chamado de Deep Learning.



Aprendizagem profunda em imagens

- Podemos aplicar uma rede neural diretamente em uma imagem!

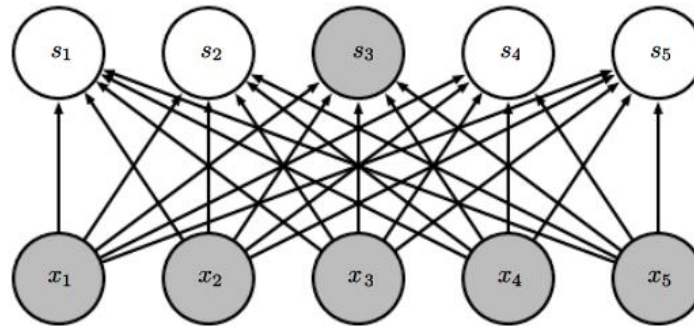


Problema: explosão do número de parâmetros que precisam ser ajustados!

Solução: redes neurais
convolucionais

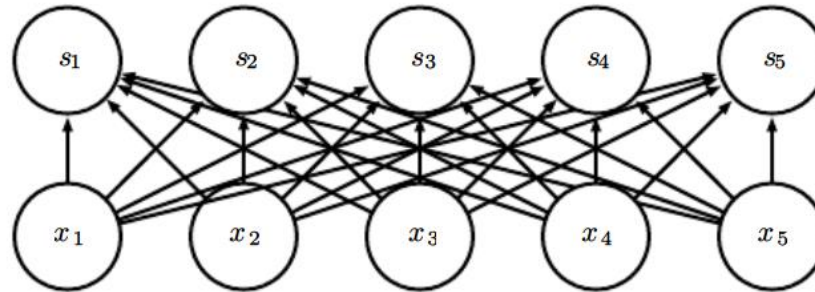
Redes neurais convolucionais

Camadas totalmente conectadas

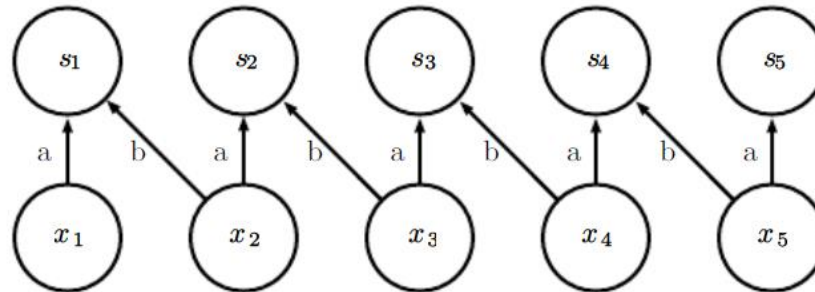


Redes neurais convolucionais

Camadas totalmente conectadas

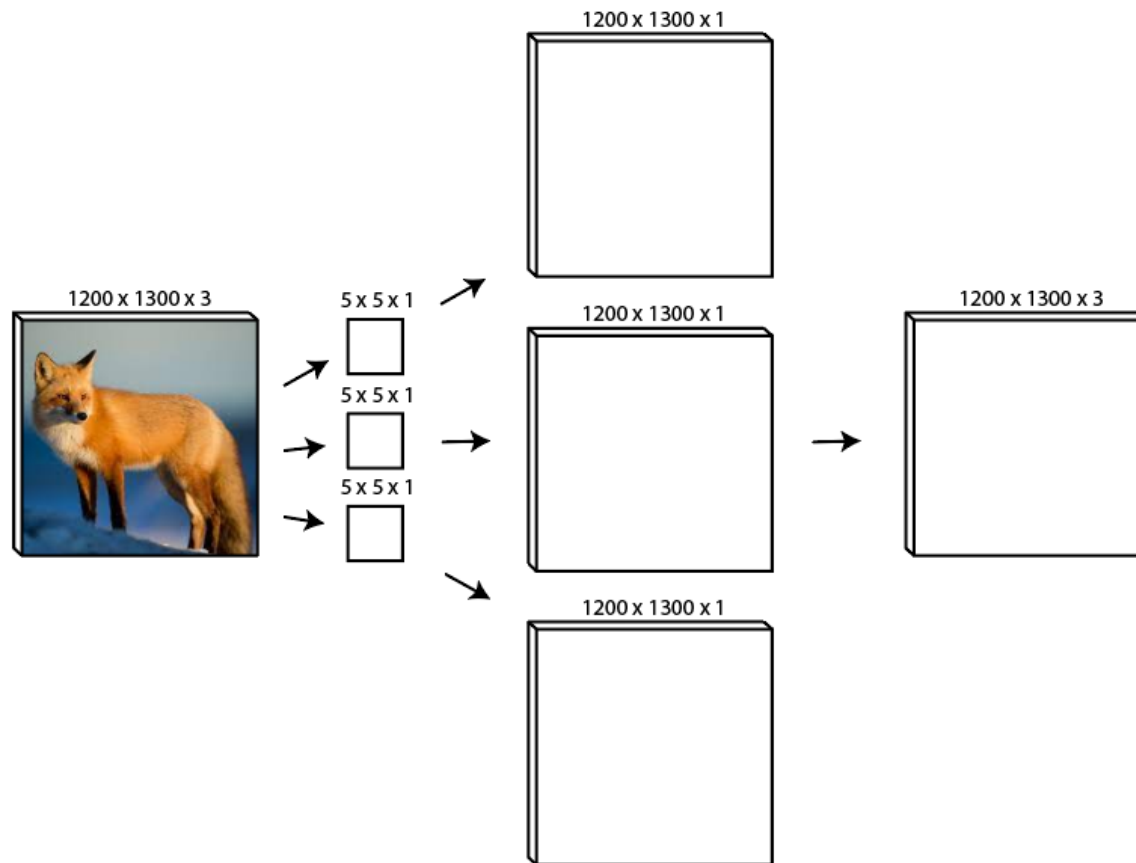


Camadas localmente conectadas com compartilhamento de parâmetros (convolução)



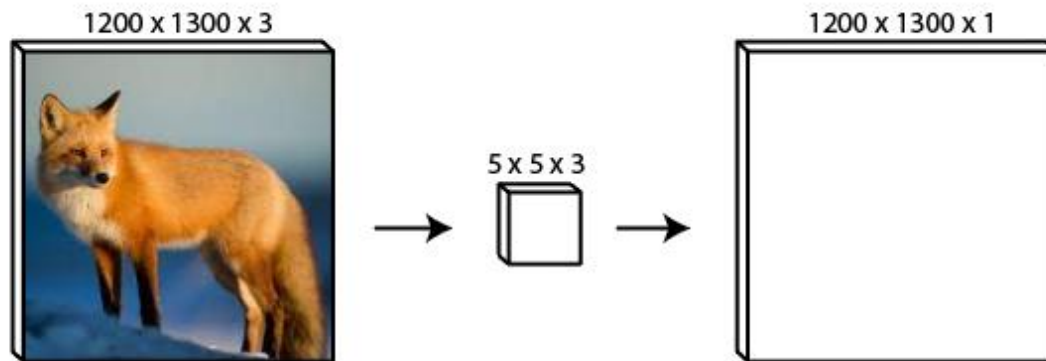
Redes neurais convolucionais (CNN)

Convolução de uma imagem colorida utilizando 3 filtros:



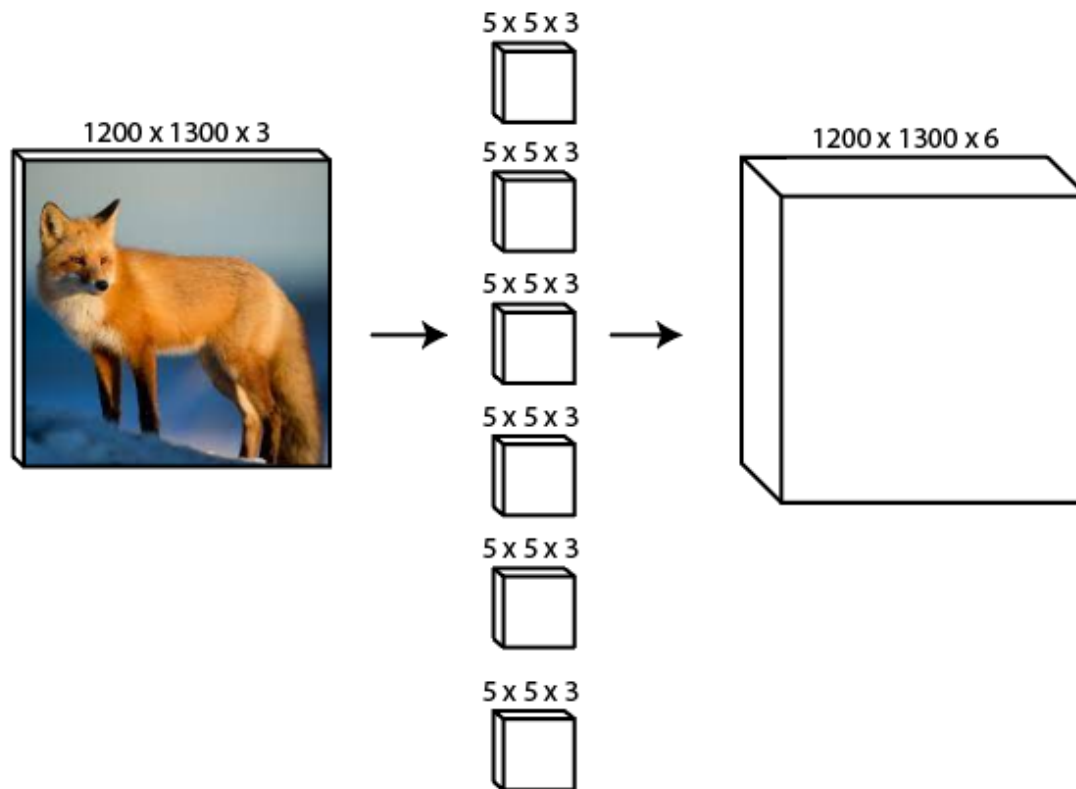
Redes neurais convolucionais (CNN)

- Ao invés de 3 filtros separados, definimos um filtro possuindo 3 canais
- Para cada posição do filtro, os $5 * 5 * 3 = 75$ valores são multiplicados e somados, gerando um valor de saída



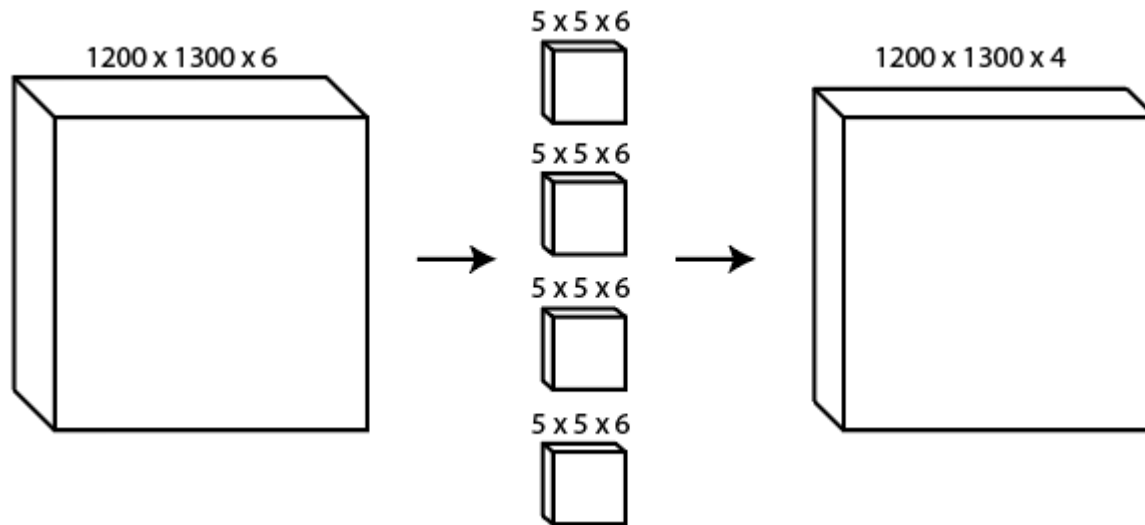
Redes neurais convolucionais (CNN)

Podemos convoluir com diversos filtros, obter várias imagens de resultado, e unir todas as imagens em uma única estrutura de saída, chamada de **tensor**



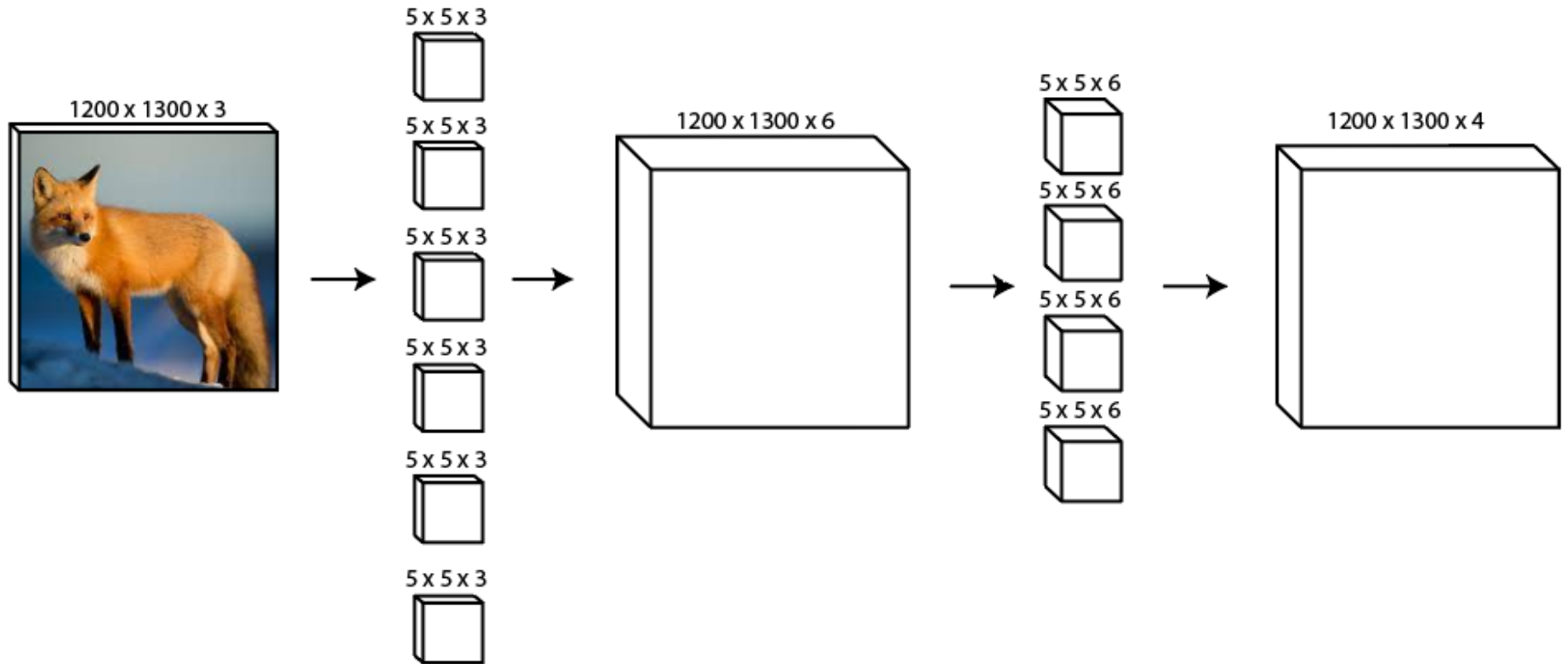
Redes neurais convolucionais (CNN)

- Novos filtros são aplicados ao resultado da camada anterior
- Note que os filtros possuem um número de canais igual ao número de canais do resultado anterior



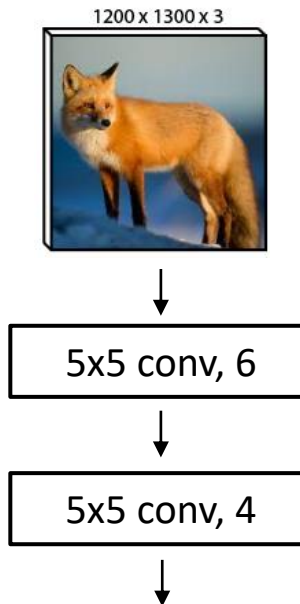
Redes neurais convolucionais (CNN)

Usualmente, uma rede é representada visualmente de forma bem enxuta...



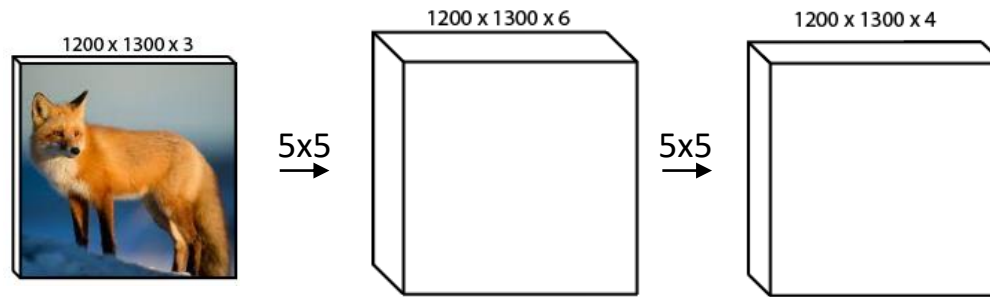
Redes neurais convolucionais (CNN)

...como retângulos que indicam o tamanho e número de filtros de cada camada



Redes neurais convolucionais (CNN)

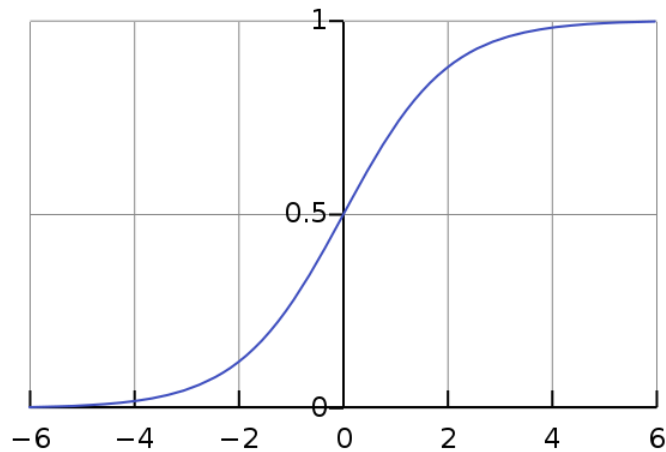
...ou pelos tamanhos dos filtros e das saídas das camadas:



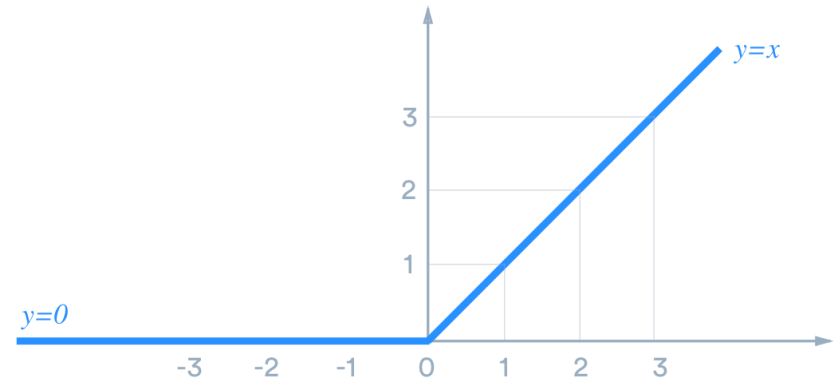
Função ReLU

Percebeu-se que ReLU funciona melhor que sigmoide como função de ativação

sigmoide

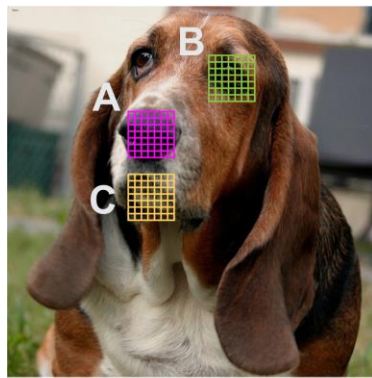


Unidade linear retificada (ReLU)

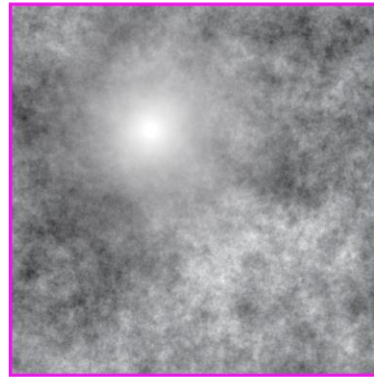


Processamento multiescala

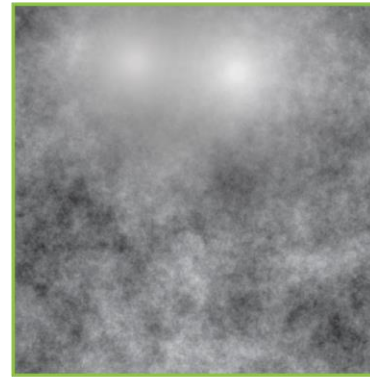
Assim como vimos no processamento multiescala de imagens, filtros pequenos possibilitam apenas identificar padrões locais em imagens



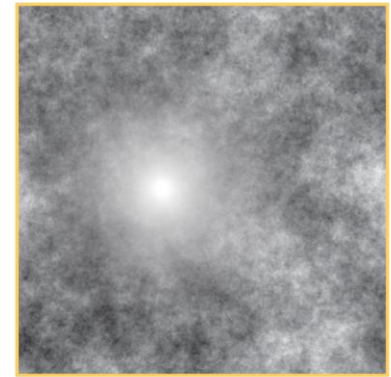
A) Nariz



B) Olho

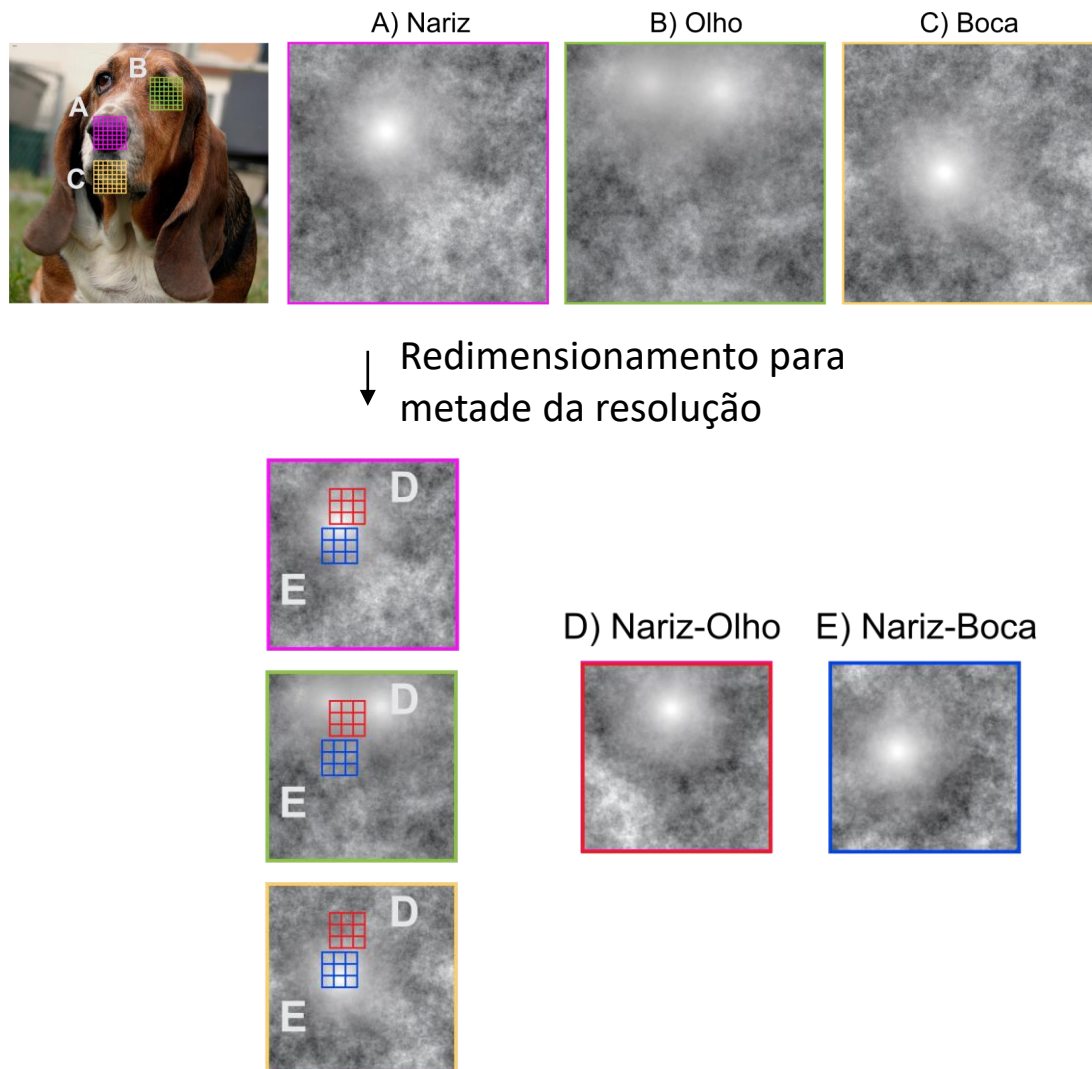


C) Boca



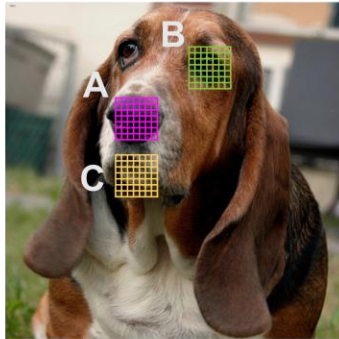
Processamento multiescala

- A redução da resolução possibilita definir filtros que operam em escalas maiores
- Adicionalmente, filtros em camadas profundas combinam atributos identificados por filtros de camadas anteriores

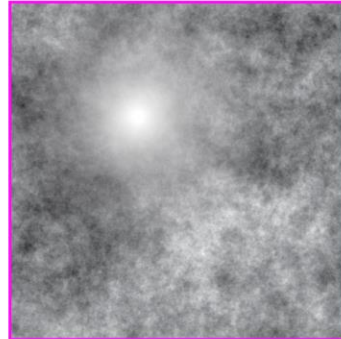


Processamento multiescala

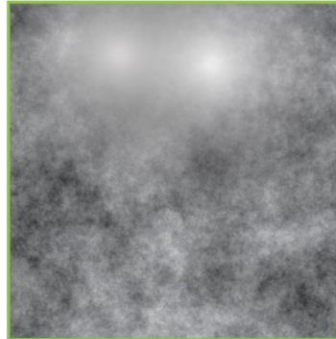
Escala 1



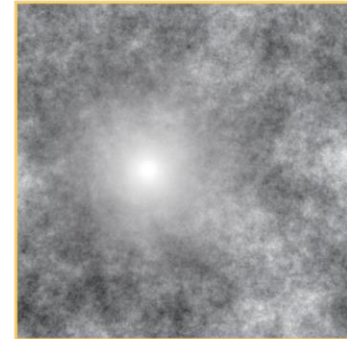
A) Nariz



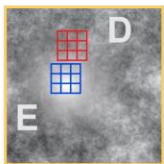
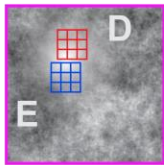
B) Olho



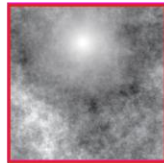
C) Boca



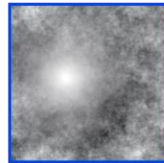
Escala 2



D) Nariz-Olho



E) Nariz-Boca



Escala 3

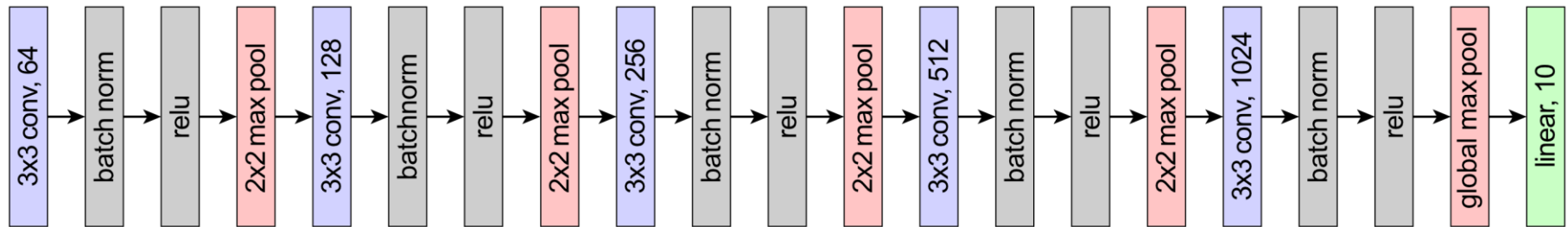


F) Focinho

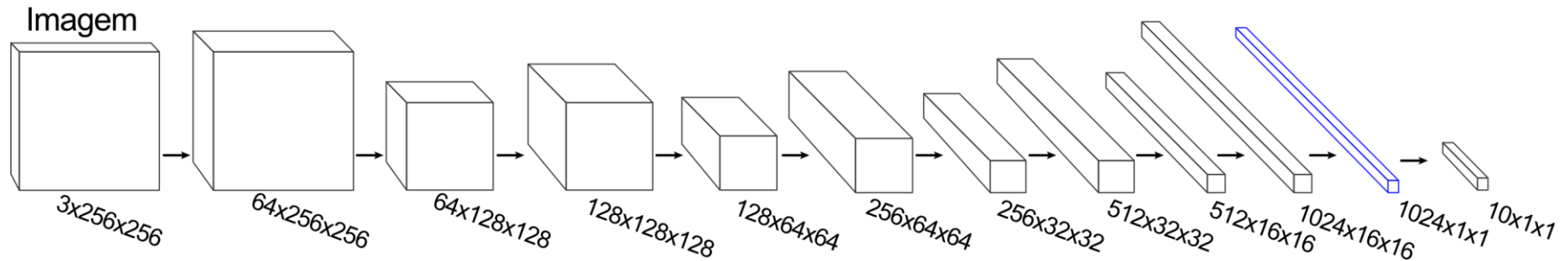


Arquitetura de CNN

- Arquitetura moderna de CNN para classificação:

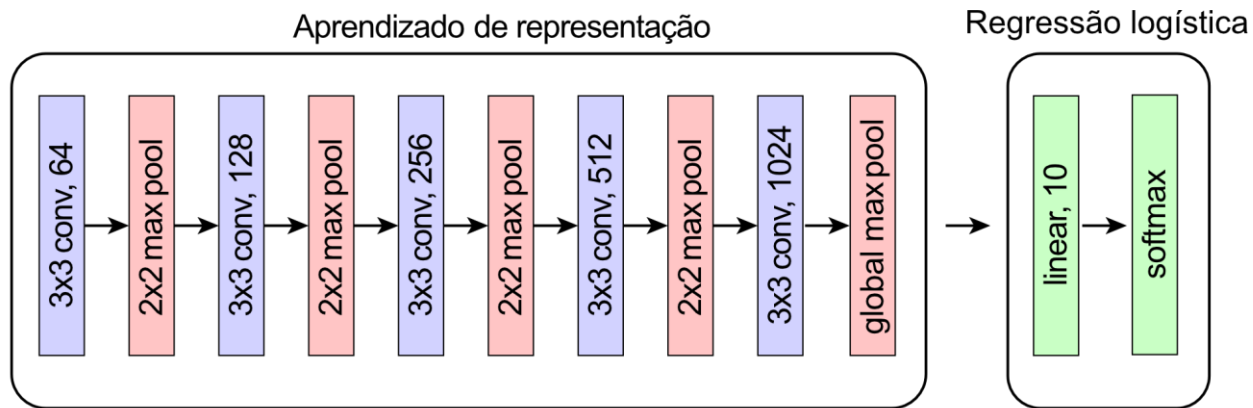


- Ativações das camadas de convolução:



Arquitetura de CNN

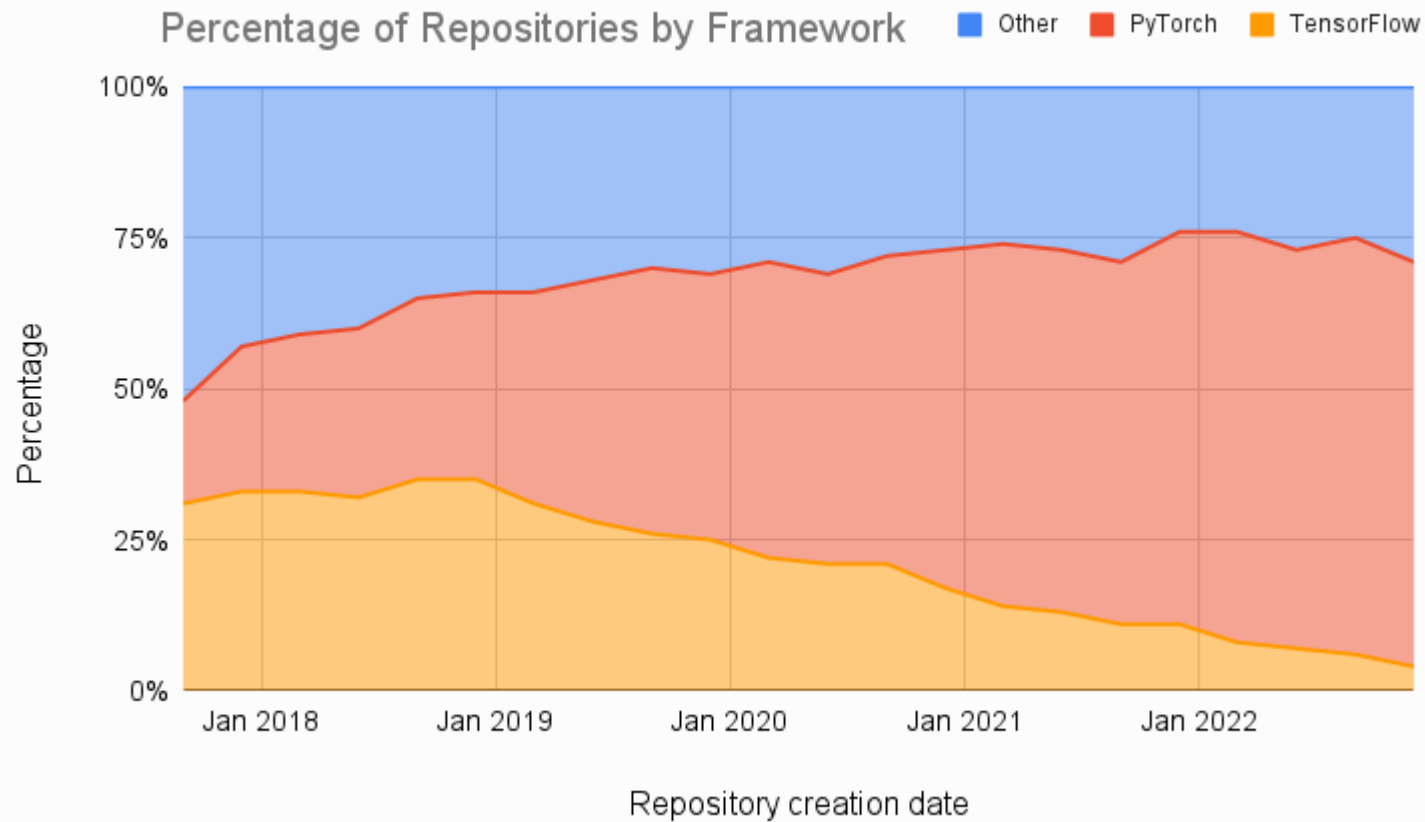
- Uma CNN é composta de duas partes principais:



Biblioteca Pytorch

- Replica as funções do numpy, com possibilidade de execução na CPU e GPU
- Principal vantagem da biblioteca: diferenciação automática
- Em 2022, se tornou parte da Linux Foundation

Biblioteca Pytorch



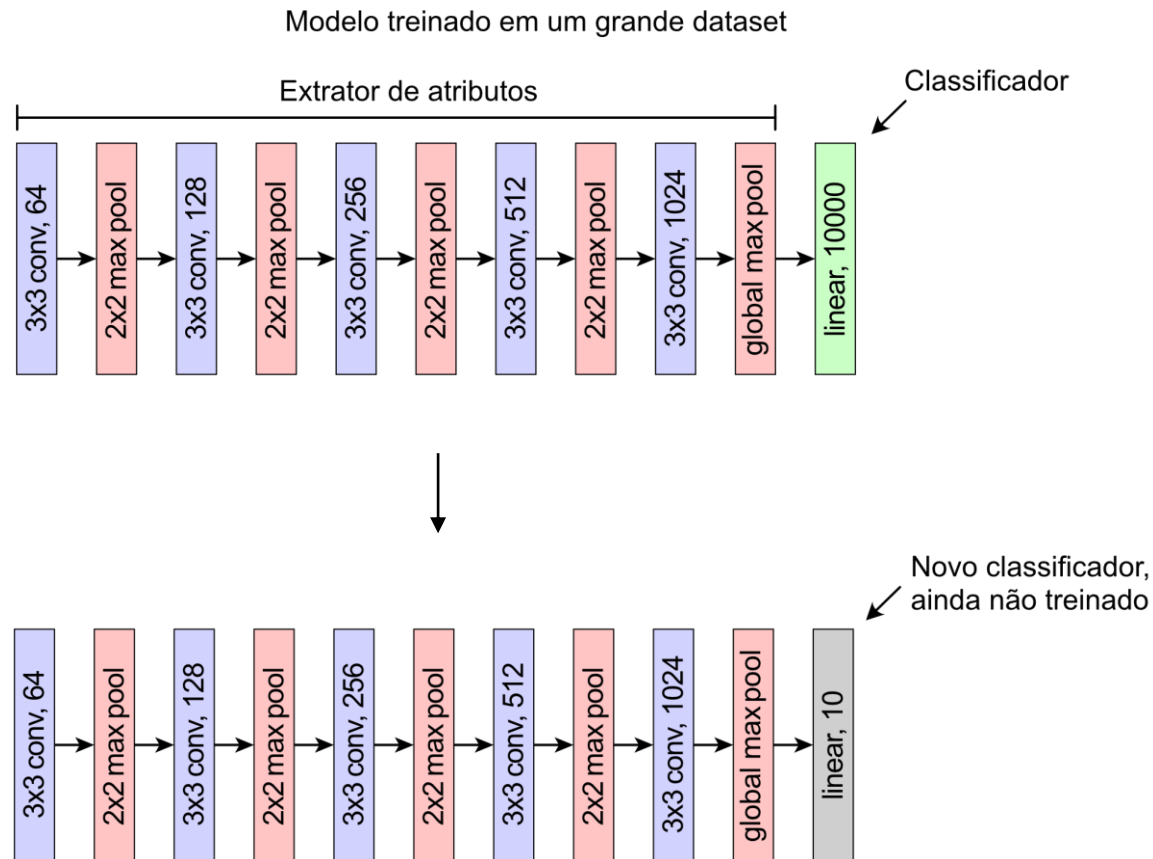
Implementação de Redes Neurais Convolucionais

Notebook 1 - Classificação de dígitos usando CNNs

Transferência de aprendizado

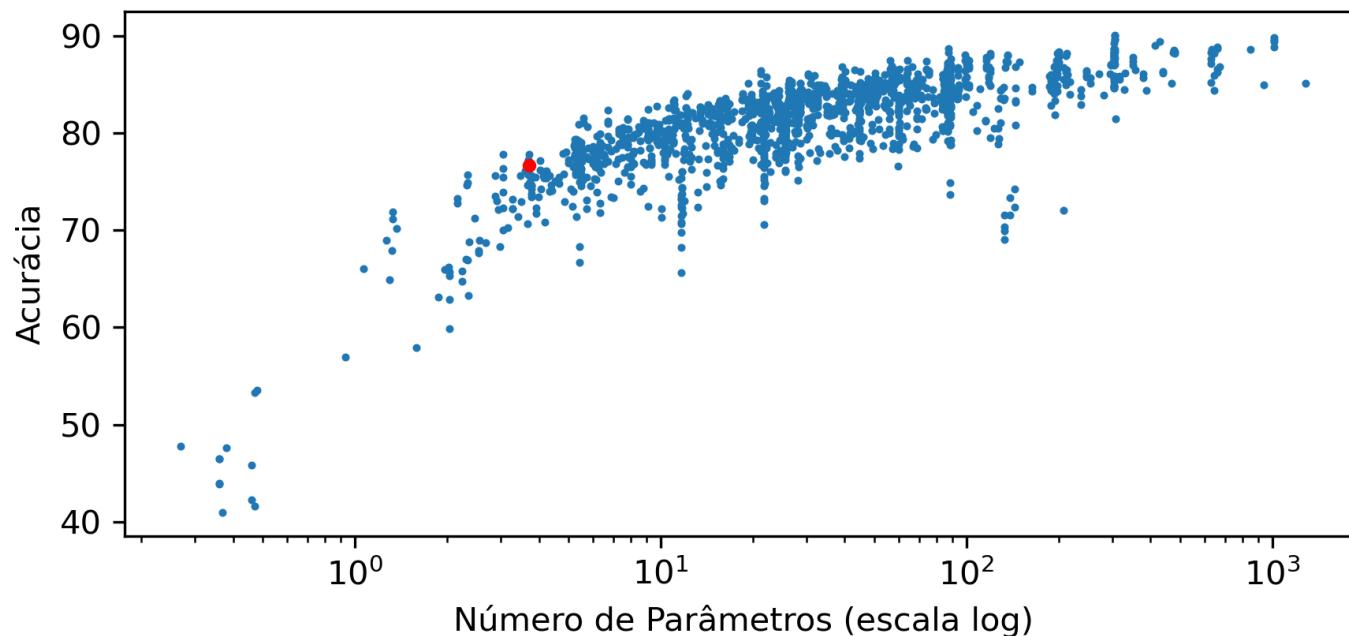
- Treinar uma pequena rede neural convolucional do zero não é difícil
- Treinar uma arquitetura de rede estado da arte é mais complicado, e custoso
- Uma estratégia usual é utilizar transferência de aprendizado, na qual é utilizado um modelo pré-treinado em um conjunto massivo de dados

Transferência de aprendizado



Pytorch Image Models (timm)

- A biblioteca timm possui uma enorme quantidade de modelos pré-treinados no conjunto de dados ImageNet
- O ImageNet possui ~ 1.3 milhões de imagens divididas em 1000 classes
- Em geral, quanto maior o modelo, melhor a acurácia:



Transferência de aprendizado

Notebooks “**2 - Classificação de imagens usando redes neurais**”